Attempt 4: What the Data Changed

From arbitrary scalar cardinalities to the real two-dimensional question

Updated next-meeting draft — not yet presented

2026-07-23

1. The Short Answer

Original question

Can one fixed-width word work better if two scalar coordinates receive arbitrary numbers of reconstruction levels?

1. We finally completed the missing GIST/CIFAR base-data comparison.
2. Arbitrary scalar cardinalities produced essentially no held-out gain.
3. A same-capacity two-dimensional codebook produced a clear gain.

Decision

The missing flexibility is the **shape of the joint codebook**, not the number of scalar levels. The original formulation stops before query testing.

2. First Avoid One Old Misunderstanding

The original Attempt 4 was **not** progressive pruning.

Original Attempt 4

Read one full word once. Change scalar level counts. Use one mixed-radix lookup.

Later prefix successor

Read a short prefix, then more bits. Add coarse filtering and accurate refinement.

Keep the conclusions separate

The prefix successor was separately closed as a direct composition of known mechanisms. Today's result concerns the original single-lookup idea.

3. Original Intuition: Use a Four-Bit Word More Fully

Two scalar coordinates share one four-bit word: 16 possible addresses.

Model	Allocation	Joint states
ordinary	2 bits + 2 bits	$4 \times 4 = 16$
ordinary	1 bit + 3 bits	$2 \times 8 = 16$
Attempt 4	arbitrary K_1, K_2	$K_1 K_2 \leq 16$

Example

$(K_1, K_2) = (3, 5)$ uses 15 addresses. Three levels on one coordinate and five on the other might fit unequal coordinate difficulty better than every power-of-two allocation.

4. Core Method: Fit, Pack, Look Up

1. Fit scalar quantizers for every permitted level count.
2. Choose K_1, K_2 with the smallest summed scalar error.
3. Pack scalar labels z_1, z_2 into one label:

```
u = z1 + K1 * z2
T[u] = distance(query pair, reconstructed pair u)
```

The scan still reads one word and performs one lookup.

The key restriction

All centers form a rectangular product $\{a_i\} \times \{b_j\}$. Changing K_1, K_2 changes the number of rows and columns; it cannot move one joint center independently.

5. Running Example: What $(3, 5)$ Can and Cannot Prove

$$x_1 \in \{-1, 0, 1\} \qquad x_2 \in \{-2, -1, 0, 1, 2\}$$

Model	States	Error
arbitrary scalar product $(3, 5)$	15	0
ordinary power-of-two product $(4, 4)$	16	positive
unrestricted 16-center 2D codebook	16	0

Claim ceiling

The example proves that power-of-two scalar allocation can waste choices. It does not prove natural-data, Recall, or speed gains. The unrestricted 2D codebook also fits exactly.

6. Four Arms Answer Four Different Questions

	Plain-language meaning	Question
`D`	power-of-two scalar grid	What does the ordinary rectangular restriction achieve?
`A`	arbitrary scalar cardinalities	Does changing only row/column counts help?
`P`	independently trained ordinary PQ	Does a standard strong implementation dominate?
`V`	same-capacity 2D vector codebook	How much is available if joint centers move freely?

Why both P and V?

P is the deployed-family control. V is the stronger opportunity test: it tells us whether the data has useful 2D structure that A fails to express.

7. What Was Actually Measured

datasets	GIST and CIFAR
fitting / held-out	8,192 / 8,192 base residuals per dataset
groups	64 fixed adjacent coordinate pairs
rates	B4 and B8
reported evidence	reconstruction, base-pair distance proxy, group prevalence, fitting and model costs

Evidence boundary

No benchmark query or ground-truth file was read. This is offline base-data evidence, not Recall or throughput evidence.

8. Main Result: A Did Not Generalize

Dataset/rate	D error removed by A	D-to-V gap closed	pair-proxy gain	groups helped
GIST B4	0%	0%	0%	0/64
GIST B8	0.0928%	0.5396%	0.1237%	7/64
CIFAR B4	0%	0%	0%	0/64
CIFAR B8	−0.0455%	−0.4186%	−0.1252%	6/64

At B4, A chose $(4, 4)$ everywhere. At B8, it chose non-power-of-two allocations in 14 GIST and 8 CIFAR groups, so the larger family was genuinely exercised.

Registered decision

Those choices still produced no material held-out benefit. Every materiality hypothesis for A failed after multiplicity correction.

9. The Important Positive Observation: Two Dimensions Help

Reduction from ordinary scalar grid to 2D VQ	B4	B8
GIST	11.4%	17.2%
CIFAR	8.8%	10.9%

All four registered tests for the existence of this opportunity passed.

Interpretation

The result is not "there is nothing to improve." Natural residual pairs contain useful 2D structure, but changing only the two scalar alphabet sizes does not capture it.

Geometric intuition

A rectangular grid can add rows or columns. A 2D codebook can place every center where the data actually needs it.

10. Lower Reconstruction Error Is Still Not Recall

lower reconstruction error

$\Downarrow$ *sometimes, not automatically*

lower query-to-database distance error

$\Downarrow$ *sometimes, not automatically*

higher Recall at matched throughput

We measured the first and a base-pair proxy for the second. A failed both.

Why query evaluation stopped

Continuing to benchmark Recall or QPS would spend effort on a formulation that already failed its cheaper discriminating test.

11. Closest Work I: PQ and OPQ

Product Quantization (PQ) splits a vector into blocks, learns one vector codebook per block, and accumulates query distance from small tables. **Optimized Product Quantization (OPQ)** jointly learns a rotation and the block codebooks:

rotate $\rightarrow$ **split into blocks** $\rightarrow$ **nearest centers** $\rightarrow$ **store labels**

Boundary

OPQ can improve which dimensions are grouped and how variance is distributed. "Learn a rotation, then quantize" is therefore not a new mechanism. OPQ is a required baseline for any future 2D model.

12. Closest Work II: AQ and CQ

Additive Quantization (AQ) reconstructs a vector as a sum:

$$x \approx c_1[i_1] + c_2[i_2] + \cdots + c_m[i_m].$$

This is expressive, but encoding and distance evaluation are harder. **Composite Quantization (CQ)** constrains interactions between dictionaries so query distances can still be evaluated from lookup tables.

Boundary

"Build one center from several learned pieces" is already covered. A new model needs a stricter database property—for example one fixed label, one lookup, or materially cheaper table construction.

13. Closest Work III: Optimize the Consumed Error

Standard PQ usually minimizes reconstruction error for individual vectors. **Pairwise**

Quantization learns a linear transform so the following quantizer better preserves pairwise squared distances or scalar products:

base-pair statistics $\rightarrow$ **learned transform** $\rightarrow$ **ordinary quantizer**

Why it matters

Our consumer is a distance estimator. Pairwise-oriented training is therefore an important baseline objective.

Claim boundary

Changing from reconstruction loss to a known pairwise loss is not itself a contribution; Recall and systems performance still have to move.

14. Closest Work IV: Systems Costs

Quicker ADC studies irregular subcode granularity, packed layouts, split lookup tables, and SIMD scanning. Nonstandard packing must beat an optimized scan. **SegPQ** compresses

PQ codebooks and supports query processing over the compressed representation. It reports up to 4.7× codebook compression with about 3.3% added query-processing overhead.

What reviewers will ask

A smaller codebook or different label layout is insufficient. Measure table construction, cache behavior, decoding, Recall, and throughput together.

ADC means query-to-code distance lookup; SIMD means one processor instruction handles several values in parallel.

15. The Original Idea Is Already Crowded

- **Muresan–Effros**: optimal 1D quantization for integer level counts.
- **Brandt**: learned scalar quantizers, integer bit allocation, packed words, tables, and ANN scan.
- **FSQ**: products of small scalar level sets, including non-power-of-two factors.
- **BAPQ**: unequal integer bit allocation among PQ subspaces.
- **Mixed-radix indexing**: standard finite-alphabet addressing.
- **Q-Palette / FibQuant**: broader dense-rate coding territory.

New empirical closure

Even where arbitrary level counts were selected, they did not recover the observed 2D opportunity.

16. A Better Modeling Question

New question

Can a compact structured 2D codebook recover most of full 2D VQ quality while preserving a fixed label and cheap lookup?

One example is a shared-shape affine model:

```
center[g,k] = mean[g] + transform[g] * shared_shape[k]
```

- one reusable non-rectangular 2D shape;
- a small scale/rotate/shear transform for each group;
- one fixed-width database label k; and
- one query table and one lookup per group.

17. Why That Is Not Yet a Contribution

At B8, a rough FP32 shared-affine representation is about 3.5 KiB; independent 2D centers use 128 KiB. But one global 128-KiB model is already small.

The systems question

Compression matters only in a setting with many local codebooks, many probed cells, or frequent updates, where table construction, cache, memory, or update cost is measured to be material.

Without a systems effect, ordinary 2D PQ/VQ is the simpler answer. SegPQ also prevents us from claiming codebook compression alone.

18. Cheapest Discriminating Next Check

Before building another ANN path:

1. fit the shared-shape affine model on the existing fitting panels;
2. evaluate it on the existing held-out panels against D and V;
3. measure how much of the D-to-V gap it recovers;
4. count model bytes and table-building arithmetic; and
5. stop if it needs nearly unrestricted per-group centers or recovers little of V's gain.

Escalation rule

Only a strong base-only result would justify a later native table-build and scan comparison. Freeze the model before touching benchmark queries.

19. Final Decision

arbitrary scalar cardinalities	`CLOSED: NO_GO_BASE_ONLY`
same-capacity 2D opportunity	real on both datasets and rates
Recall / QPS	not measured
prefix successor	separately closed as direct composition
structured 2D successor	a new question, not yet established

Meeting takeaway

The original (K_1, K_2) idea is not the right model for natural residuals. Evidence points to joint 2D geometry, but a future direction must also move a real database-systems cost frontier.

References and Evidence I

- Muresan and Effros. "Quantization as Histogram Segmentation." DCC 2002.
- Brandt. "Transform Coding for Fast Approximate Nearest Neighbor Search in High Dimensions." CVPR 2010.
- Ge et al. "Optimized Product Quantization." CVPR 2013. https://www.microsoft.com/en-us/research/wp-content/uploads/2013/06/cvpr13opq.pdf
- Babenko and Lempitsky. "Additive Quantization for Extreme Vector Compression." CVPR 2014. https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html
- Zhang, Du, and Wang. "Composite Quantization for Approximate Nearest Neighbor Search." ICML 2014. https://proceedings.mlr.press/v32/zhangd14.html
- Guo et al. "Adaptive Bit Allocation Product Quantization." Neurocomputing, 2016. https://doi.org/10.1016/j.neucom.2015.07.062
- Babenko, Arandjelovic, and Lempitsky. "Pairwise Quantization." 2016. https://arxiv.org/abs/1606.01550
- Mentzer et al. "Finite Scalar Quantization." ICLR 2024. https://arxiv.org/abs/2309.15505
- André, Kermarrec, and Le Scouarnec. "Quicker ADC." TPAMI. https://arxiv.org/abs/1812.09162
- Liu et al. "Not Small Enough? SegPQ." PVLDB 2025. https://www.vldb.org/pvldb/vol18/p3730-liu.pdf
- Lee and Song. "Q-Palette." 2025. https://arxiv.org/abs/2509.20214
- Lee and Kim. "FibQuant." 2026. https://arxiv.org/abs/2605.11478
- Riskin et al. "Index Assignment for Progressive Transmission of Full Search Vector Quantization." IEEE TIP, 1994.
- André et al. "Derived Codebooks for High-Accuracy Nearest Neighbor Search." 2019.
- Douze et al. "Polysemous Codes." ECCV 2016.
- Base-only result: `saq-a4-original-reopening-protocol@5503e87`, `docs/saq_a4_or_b_base_only_result_2026_07_23.md`.
- R0: `saq-a4-r0-static-gate@617ad25`. Prefix S0: `saq-a4-prefix-novelty-gate@7f4c001`.